

Express.js & REST API

Membangun Backend Server: Routing, Middleware, dan CRUD

Pertemuan · Lanjutan dari Pertemuan 2 (HTML/CSS/JS)

Agenda Pertemuan 3

REST → Express Setup → Routing → Middleware → Praktik

SEKSI 1

REST & HTTP

- Konsep REST & arsitektur client-server
- HTTP Methods: GET/POST/PUT/PATCH/DELETE
- Status Code penting (2xx, 4xx, 5xx)
- URL pattern & JSON request/response

SEKSI 2

Express.js

- Setup project & install dependencies
- Hello World & routing dasar
- Route params, query string, POST body
- Struktur project yang rapi

SEKSI 3

Middleware

- Konsep middleware pipeline
- CORS, express.json(), body-parser
- Auth guard & custom middleware
- Error handler 4-parameter

SEKSI 4

Praktik

- CRUD produk in-memory (array)
- Router modular (pisah file)
- Validasi input + search & filter
- Testing dengan Thunder Client

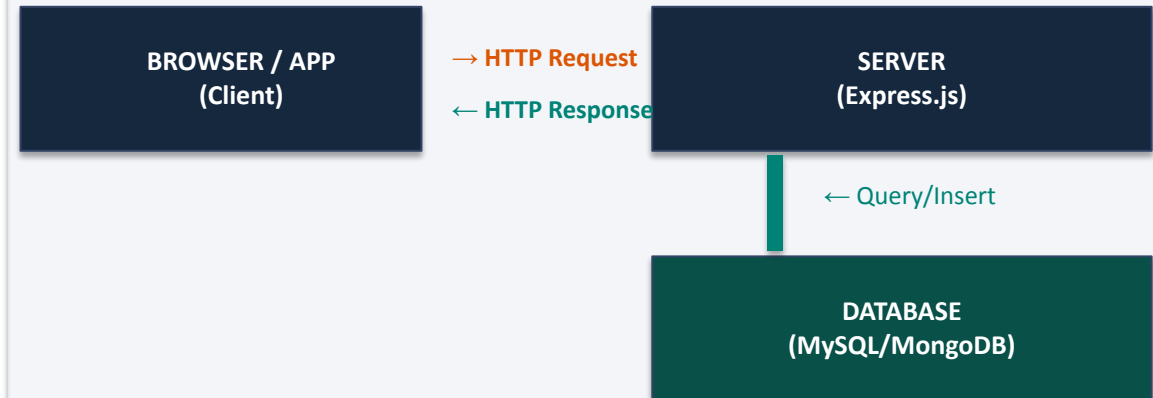
REST & HTTP

Arsitektur komunikasi client-server standar web modern

6 Prinsip REST (Roy Fielding, 2000)

- **1. Client-Server**
UI dan data dipisah. Frontend tidak tahu bagaimana data disimpan.
- **2. Stateless**
Setiap request harus lengkap sendiri. Server tidak menyimpan session.
- **3. Cacheable**
Response bisa di-cache. Header Cache-Control mengaturnya.
- **4. Layered System**
Client tidak tahu apakah bicara langsung ke server atau proxy.
- **5. Uniform Interface**
Satu cara untuk semua resource: pakai URL + HTTP Method.
- **6. Code on Demand**
(Opsional) Server bisa kirim kode yang dieksekusi client.

Analogi: REST seperti Menu Restoran



Request contoh:

`GET /api/products/5`

Response contoh:

`{ "id": 5, "nama": "Apel", "harga": 5000 }`

Format data:

JSON (JavaScript Object Notation)

Protocol:

HTTP / HTTPS

Catatan: REST bukan sebuah library atau tool. Ini adalah arsitektur/konvensi. Express.js membantu kita membangun server yang mengikuti konvensi REST.

HTTP Methods & Kapan Menggunakannya

Method	Fungsi	Status	Contoh URL	Catatan Penting
GET	Ambil / baca data	200 OK	GET /products GET /products/5	Tidak mengubah data sama sekali. Boleh di-cache. Idempotent.
POST	Buat data baru	201 Created	POST /products (body: {nama,harga})	Data ada di request body. Setiap call buat data baru.
PUT	Update data penuh	200 OK	PUT /products/5 (body: semua field)	Ganti SELURUH resource. Kalau ada field yang tidak dikirim → hilang.
PATCH	Update sebagian	200 OK	PATCH /products/5 (body: {harga:6000})	Ganti field TERTENTU saja. Lebih efisien dari PUT.
DELETE	Hapus data	204 No Content	DELETE /products/5	Tidak ada response body. Status 204 = sukses tanpa isi.

Tip: Ingat pakai method yang benar. Salah method bisa membuat API susah dipahami dan melanggar konvensi REST. GET tidak boleh mengubah data. POST tidak cocok untuk update. Ini penting untuk kerja sama tim frontend-backend.

2xx — Sukses

200 OK

Request berhasil. Paling umum untuk GET, PUT, PATCH.

201 Created

Data baru berhasil dibuat. Pakai untuk response POST.

204 No Content

Sukses tapi tidak ada data yang dikembalikan. Untuk DELETE.

3xx — Redirect

301 Moved Permanently

Resource pindah permanen ke URL baru.

302 Found

Redirect sementara.

304 Not Modified

Cache masih valid, tidak perlu kirim ulang data.

4xx — Client Error

400 Bad Request

Data yang dikirim client salah / tidak lengkap.

401 Unauthorized

Belum login / token tidak valid.

403 Forbidden

Sudah login tapi tidak punya akses ke resource ini.

404 Not Found

Resource tidak ditemukan di server.

422 Unprocessable

Data valid tapi gagal diproses (validasi business logic).

5xx — Server Error

500 Internal Server Error

Ada bug di server. Stack trace ada di log, bukan di response!

502 Bad Gateway

Server upstream tidak merespons.

503 Service Unavailable

Server down / maintenance / overload.

Aturan Penamaan URL REST

- **Gunakan kata benda (noun), bukan kata kerja**
GET /products ✓ *GET /getProducts* ✗
- **Jamak untuk collection, tunggal tidak dipakai**
GET /products ✓ *GET /product* ✗
- **Huruf kecil semua, pisah dengan tanda hubung**
/product-categories ✓ */ProductCategories* ✗
- **Hierarki resource dengan slash**
/users/5/orders → *order milik user 5*
- **ID resource ada di URL, bukan query string**
GET /products/5 ✓ *GET /products?id=5* ✗
- **Query string untuk filter, sort, paginasi**
GET /products?page=1&limit=10&sort=harga

Contoh Lengkap: Resource */api/products*

GET	/api/products	Semua produk
GET	/api/products?search=apel	Filter by nama
GET	/api/products?page=1&limit=5	Paginasi
GET	/api/products/:id	Satu produk
POST	/api/products	Buat produk baru
PUT	/api/products/:id	Update penuh
PATCH	/api/products/:id	Update sebagian
DELETE	/api/products/:id	Hapus produk
GET	/api/products/:id/reviews	Review milik produk
POST	/api/products/:id/reviews	Tambah review

Express.js

Framework Node.js terpopuler untuk HTTP server

Setup Project Express.js

Terminal / Command Prompt

```
# Buat folder project baru
mkdir my-api && cd my-api

# Inisialisasi package.json
npm init -y

# Install dependencies utama
npm install express cors

# Install dev dependency
npm install -D nodemon

# Cek versi Node (minimal v18)
node -v
npm -v
```

package.json

```
// package.json
// Tambahkan ini di bagian "scripts":
{
  "name": "my-api",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "start": "node index.js",
    "dev": "nodemon index.js"
  },
}

# Jalankan dengan hot-reload (development):
npm run dev

# Jalankan tanpa hot-reload (production):
npm start
```

```
my-api/
├── index.js      ← file utama
├── routes/
│   └── products.js
├── middleware/
│   └── logger.js
├── package.json
└── .gitignore
```

index.js

```
// index.js - Hello World Server
const express = require('express');
const app = express();

// Middleware bawaan Express: parse JSON otomatis
app.use(express.json());
app.use(require('cors')());

// Route pertama kita
app.get('/', (req, res) => {
  res.json({ message: 'Hello World!' });
});

// Jalankan server di port 3000
app.listen(3000, () => {
  console.log('Server berjalan di port 3000');
});
```

Routing: req.params, req.query, req.body

index.js

```
// 1. ROUTE BIASA — tidak ada parameter
app.get('/products', (req, res) => {
  res.json({ message: 'daftar semua produk' });
});

// 2. ROUTE PARAMS — :id dari URL
// URL: GET /products/42 → req.params.id = "42"
app.get('/products/:id', (req, res) => {
  const id = parseInt(req.params.id);
  if (isNaN(id)) return res.status(400).json({ error: 'ID harus angka' });
  res.json({ id, pesan: 'data produk' });
});

// 3. QUERY STRING — GET /products?page=1&limit=5&search=apel
app.get('/products', (req, res) => {
  const { page = 1, limit = 10, search = '' } = req.query;
  res.json({ page: Number(page), limit: Number(limit), search });
});

// 4. POST BODY — data dikirim di body request
// Butuh app.use(express.json()) agar req.body terisi!
app.post('/products', (req, res) => {
  const { nama, harga, stok } = req.body;
  if (!nama || !harga) return res.status(400).json({ error: 'nama & harga wajib' });
  res.status(201).json({ id: 1, nama, harga, stok });
});
```

req.params

Nilai dari URL path (:id, :slug, dll). Selalu string — perlu konversi ke Number jika dipakai sebagai angka.

req.query

Nilai setelah ? di URL (?page=1&limit=10). Dipakai untuk filter, sort, paginasi. Tidak wajib ada.

req.body

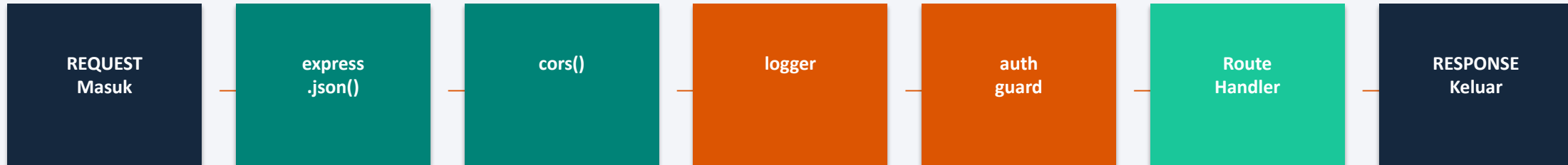
Data dari POST/PUT/PATCH. WAJIB ada `app.use(express.json())` sebelum route, atau `req.body` akan undefined!

Headers: `req.headers["authorization"]` — dipakai untuk token auth.

Middleware & Error Handling

Fungsi yang berjalan antara request masuk dan response keluar

Middleware — Pipeline Request ke Response



KUNCI MIDDLEWARE: Setiap fungsi middleware WAJIB memanggil `next()` agar request bisa lanjut ke middleware berikutnya. Kalau `next()` tidak dipanggil → request menggantung (hang) dan client tidak dapat response.

Anatomi Middleware

```
// Anatomi sebuah middleware function
function namaMiddleware (req, res, next) {
  // ... lakukan sesuatu ...
  next(); // ← WAJIB! Lanjutkan ke middleware berikutnya
}

// Cara pakai: GLOBAL (semua route kena)
app.use(namaMiddleware);

// Cara pakai: SPESIFIK satu route saja
app.get('/admin', auth, namaMiddleware, handlerFn);

// Logger sederhana - contoh custom middleware
function logger (req, res, next) {
  const start = Date.now();
  res.on('finish', () => {
    console.log(`${req.method} ${req.url}` + res.statusCode +
      (Date.now()-start)+'ms');
  });
  next();
}
```

Auth Guard Middleware

```
// Middleware yang MENGHENTIKAN request
// (tidak memanggil next())
function auth (req, res, next) {
  const token = req.headers.authorization;

  if (!token) {
    return res.status(401).json({
      error: 'Token tidak ada, login dulu!'
    });
    // ← TIDAK ada next() di sini
  }

  // Token ada → lanjut
  req.user = decodeToken(token);
  next();
}

// Pasang ke route yang butuh login
app.get('/profile', auth, (req, res) => {
  res.json(req.user);
});
```

CORS & Error Handler — Dua Middleware Wajib

CORS Setup

```
// — CORS (Cross-Origin Resource Sharing) —  
// CORS = izin browser dari domain A akses API di domain B  
// Tanpa cors() → browser blokir request dari localhost:5173  
  
const express = require('express');  
const cors    = require('cors');  
const app     = express();  
  
// Urutan middleware SANGAT penting!  
app.use(cors()); // 1. CORS dulu  
app.use(express.json()); // 2. Parse JSON body  
app.use(express.urlencoded({extended: true})); // 3. Parse form data  
  
// CORS dengan konfigurasi spesifik (production)  
app.use(cors({  
  origin: ['http://localhost:3000', 'https://myapp.com'],  
  methods: ['GET', 'POST', 'PUT', 'DELETE'],  
  credentials: true, // Izinkan cookies/auth headers  
}));
```

Error Handler (wajib paling bawah)

```
// — ERROR HANDLER 4-Parameter —  
// WAJIB 4 param: (err, req, res, next)  
// HARUS di pasang PALING BAWAH dari semua middleware  
  
app.use((err, req, res, next) => {  
  console.error(err.stack); // log di server, bukan ke client  
  
  res.status(err.status || 500).json({  
    error: err.message || 'Internal Server Error'  
  });  
});  
  
// Cara lempar error dari route ke error handler:  
app.get('/produk/:id', async (req, res, next) => {  
  try {  
    const p = await Produk.findById(req.params.id);  
    if (!p) { const e = new Error('Produk tidak ditemukan');  
              e.status = 404; throw e; }  
    res.json(p);  
  } catch (e) { next(e); } // kirim ke error handler!  
});
```



PERHATIAN: Error handler HARUS memiliki tepat 4 parameter (err, req, res, next). Kalau hanya 3 parameter, Express tidak mengenalinya sebagai error handler dan error tidak akan tertangkap! Selalu pasang di baris PALING BAWAH sebelum app.listen().

Praktik

Tiga latihan: sederhana → modular → CRUD + validasi

Terminal — Setup

```
# Setup project baru
mkdir produk-api && cd produk-api
npm init -y
npm install express cors
npm install -D nodemon

# Tambah ke package.json → scripts:
"dev": "nodemon index.js"
```

index.js — Data awal

```
// index.js — Setup awal
const express = require('express');
const app = express();

app.use(express.json());
app.use(require('cors')());

// — DATA AWAL (simulasi database) —
let products = [
  { id: 1, nama: 'Apel Fuji', kategori: 'buah', harga: 15000, stok: 100 },
  { id: 2, nama: 'Jeruk Mandarin', kategori: 'buah', harga: 8000, stok: 50 },
  { id: 3, nama: 'Wortel', kategori: 'sayur', harga: 5000, stok: 200 },
  { id: 4, nama: 'Bayam', kategori: 'sayur', harga: 3000, stok: 150 },
  { id: 5, nama: 'Susu Ultra', kategori: 'minuman', harga: 18000, stok: 30 },
];
let nextId = 6; // counter auto-increment
```

Struktur Data Produk

id	Number	Auto-increment, primary key
nama	String	Nama produk, wajib ada
kategori	String	buah / sayur / minuman
harga	Number	Harga dalam Rupiah, wajib ada
stok	Number	Jumlah stok, default 0

index.js — GET & POST routes

```
// — GET /products — ambil semua produk —————
app.get('/products', (req, res) => {
  res.json(products);
});

// — GET /products/:id — satu produk —————
app.get('/products/:id', (req, res) => {
  const p = products.find(p => p.id === req.params.id);
  if (!p) return res.status(404).json({ error: 'Produk tidak ditemukan' });
  res.json(p);
});

// — POST /products — buat produk baru —————
app.post('/products', (req, res) => {
  const { nama, kategori, harga, stok = 0 } = req.body;

  // Validasi field wajib
  if (!nama) return res.status(400).json({ error: 'nama wajib diisi' });
  if (!harga) return res.status(400).json({ error: 'harga wajib diisi' });
  if (harga <= 0 ) return res.status(400).json({ error: 'harga harus positif' });

  const produkBaru = { id: nextId++, nama, kategori, harga, stok,
    createdAt: new Date().toISOString() };
  products.push(produkBaru);
  res.status(201).json(produkBaru);
});
```

Contoh Request & Response

200 GET /products

```
[
  { "id":1,"nama":"Apel Fuji",
    "harga":15000,"stok":100 },
  ...
]
```

200 GET /products/2

```
{ "id":2, "nama":"Jeruk Mandarin",
  "harga":8000, "stok":50 }
```

404 GET /products/99

```
{ "error": "Produk tidak ditemukan" }
```

201 POST /products
body: {nama, harga}

```
{ "id":6, "nama":"Mangga",
  "harga":20000, "stok":0,
  "createdAt": "2026-..." }
```

400 POST /products
body: {} (kosong)

```
{ "error": "nama wajib diisi" }
```


index.js — PUT, PATCH, DELETE

```
// — PUT /products/:id — update penuh —————
app.put('/products/:id', (req, res) => {
  const idx = products.findIndex(p => p.id == req.params.id);
  if (idx === -1) return res.status(404).json({ error: 'Not found' });

  // Spread: ambil data lama, timpa dengan data baru
  products[idx] = { ...products[idx], ...req.body, id: +req.params.id };
  res.json(products[idx]);
});

// — PATCH /products/:id — update sebagian —————
// PATCH sama seperti PUT tapi lebih eksplisit tentang partial update
app.patch('/products/:id', (req, res) => {
  const idx = products.findIndex(p => p.id == req.params.id);
  if (idx === -1) return res.status(404).json({ error: 'Not found' });
  Object.assign(products[idx], req.body); // merge partial update
  res.json(products[idx]);
});

// — DELETE /products/:id — hapus produk —————
app.delete('/products/:id', (req, res) => {
  const before = products.length;
  products = products.filter(p => p.id != req.params.id);
  if (products.length === before) // panjang tidak berubah = tidak ada yang
    dihapus
    return res.status(404).json({ error: 'Not found' });
  res.status(204).send(); // 204 = sukses, tidak ada body
});

// — Jalankan server —————
app.listen(3000, () => console.log('API jalan di http://localhost:3000'));
```

Testing dengan Thunder Client

GET	/products — (kosong)
GET	/products/2 — (kosong)
POST	/products {nama,harga,stok}
PUT	/products/1 {nama,harga,stok}
PATCH	/products/1 {harga:20000}
DELETE	/products/3 — (kosong)

Struktur Folder Modular

Struktur project setelah refactor

produk-api-2/

```
|— index.js
|— routes/
|   |— products.js
|   |— users.js
|— middleware/
|   |— logger.js
|   |— validateProduct.js
|— data/
|   |— seed.js
|— package.json
```

index.js

```
// index.js - hanya konfigurasi & mount router
const express = require('express');
const cors = require('cors');
const logger = require('./middleware/logger');
const app = express();
app.use(cors());
app.use(express.json());
app.use(logger); // semua route kena logger

// Mount routers ke base path
app.use('/api/products', require('./routes/products'));
app.use('/api/users', require('./routes/users'));

// Error handler - selalu paling bawah
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(err.status || 500).json({ error: err.message });
});

app.listen(3000, () => console.log('API jalan di http://localhost:3000'));
```

data/seed.js

```
// data/seed.js - data awal simulasi database
```

```
module.exports = [
```

```
{ id: 1, nama: 'Apel Fuji',      kategori: 'buah',    harga: 15000, stok: 100 },
```

```
{ id: 2, nama: 'Jeruk Mandarin', kategori: 'buah',    harga: 8000, stok: 50 },
```

```
{ id: 3, nama: 'Wortel',        kategori: 'sayur',   harga: 5000, stok: 200 },
```

```
{ id: 4, nama: 'Bayam',         kategori: 'sayur',   harga: 3000, stok: 150 },
```

```
{ id: 5, nama: 'Susu Ultra',    kategori: 'minuman', harga: 18000, stok: 30 },
```

```
];
```

middleware/logger.js

```
// middleware/logger.js - log setiap request masuk
function logger(req, res, next) {
  const now = new Date().toISOString();
  console.log(`[${now}] ${req.method} ${req.originalUrl}`);
  next();
}

module.exports = logger;
```

middleware/validateProduct.js

```
// middleware/validateProduct.js - validasi & sanitasi body produk
function validateProduct(req, res, next) {
  const { nama, harga, stok } = req.body;
  const errors = [];

  if (!nama || nama.trim().length < 2)
    errors.push('nama wajib, minimal 2 karakter');

  if (!harga || isNaN(harga) || Number(harga) <= 0)
    errors.push('harga wajib, harus angka positif');

  if (stok !== undefined && (isNaN(stok) || Number(stok) < 0))
    errors.push('stok harus angka >= 0');

  if (errors.length > 0) {
    return res.status(400).json({ errors });
  }

  // Sanitasi: pastikan tipe data konsisten
  req.body.nama = nama.trim();
  req.body.harga = Number(harga);
  req.body.stok = stok !== undefined ? Number(stok) : 0;

  next();
}

module.exports = validateProduct;
```

routes/products.js

```
// routes/products.js - semua route /api/products
const router = require('express').Router();
const validate = require('../middleware/validateProduct');
// Data lokal di router ini
let products = require('../data/seed');
let nextId = products.length + 1;

router.get('/', (req, res) => {
  let result = [...products];

  if (req.query.kategori)
    result = result.filter(p => p.kategori ===
req.query.kategori);

  if (req.query.search)
    result = result.filter(p =>
p.nama.toLowerCase().includes(req.query.search.toLowerCase()));

  if (req.query.minHarga)
    result = result.filter(p => p.harga >= +req.query.minHarga);
  if (req.query.maxHarga)
    result = result.filter(p => p.harga <= +req.query.maxHarga)
  if (req.query.sort === 'harga')
    result.sort((a, b) => a.harga - b.harga);
  else if (req.query.sort === 'nama')
    result.sort((a, b) => a.nama.localeCompare(b.nama));
```

routes/products.js

```
res.json({ total: result.length, data: result });
});

router.get('/:id', (req, res) => {
  const p = products.find(p => p.id === req.params.id);
  if (!p) return res.status(404).json({ error: 'Produk tidak
ditemukan' });
  res.json(p);
});

router.post('/', validate, (req, res) => {
  const p = { id: nextId++, ...req.body, createdAt: new
Date().toISOString() };
  products.push(p);
  res.status(201).json(p);
});

router.put('/:id', validate, (req, res) => {
  const idx = products.findIndex(p => p.id === req.params.id);
  if (idx === -1) return res.status(404).json({ error: 'Produk
tidak ditemukan' });
  products[idx] = { ...products[idx], ...req.body, id:
+req.params.id };
  res.json(products[idx]);
});
```

```
routes/products.js
```

```
// PATCH /api/products/:id - update sebagian
router.patch('/:id', (req, res) => {
  const idx = products.findIndex(p => p.id == req.params.id);
  if (idx === -1) return res.status(404).json({ error: 'Produk
tidak ditemukan' });
  Object.assign(products[idx], req.body);
  res.json(products[idx]);
});

// DELETE /api/products/:id - hapus produk
router.delete('/:id', (req, res) => {
  const before = products.length;
  products = products.filter(p => p.id != req.params.id);
  if (products.length === before)
    return res.status(404).json({ error: 'Produk tidak ditemukan'
});
  res.status(204).send();
});

module.exports = router;
```


routes/users.js

```
// routes/users.js - semua route /api/users
const router = require('express').Router();

let users = [
  { id: 1, nama: 'Agus', email: 'agus@tazkia.ac.id', role: 'admin' },
  { id: 2, nama: 'Budi', email: 'budi@tazkia.ac.id', role: 'user' },
  { id: 3, nama: 'Citra', email: 'citra@tazkia.ac.id', role: 'user' },
];

let nextId = users.length + 1;

router.get('/', (req, res) => {
  const safe = users.map(({ id, nama, email, role }) => ({ id, nama, email, role }));
  res.json({ total: safe.length, data: safe });
});

router.get('/:id', (req, res) => {
  const u = users.find(u => u.id == req.params.id);
  if (!u) return res.status(404).json({ error: 'User tidak ditemukan' });
  res.json(u);
});

router.post('/', (req, res) => {
  const { nama, email, role = 'user' } = req.body;
  const errors = [];
```

routes/users.js

```
if (!nama || nama.trim().length < 2)
  errors.push('nama wajib, minimal 2 karakter' );
if (!email || !email.includes('@'))
  errors.push('email wajib dan harus valid' );
if (!['admin', 'user'].includes(role))
  errors.push('role harus "admin" atau "user"' );

if (errors.length > 0) return res.status(400).json({ errors });

const u = { id: nextId++, nama: nama.trim(), email, role };
users.push(u);
res.status(201).json(u);
});

router.put('/:id', (req, res) => {
  const idx = users.findIndex(u => u.id == req.params.id);
  if (idx === -1) return res.status(404).json({ error: 'User tidak ditemukan' });
  users[idx] = { ...users[idx], ...req.body, id: +req.params.id };
  res.json(users[idx]);
});

router.delete('/:id', (req, res) => {
  const before = users.length;
  users = users.filter(u => u.id != req.params.id);
  if (users.length === before)
    return res.status(404).json({ error: 'User tidak ditemukan' });
  res.status(204).send();
});

module.exports = router;
```

PRODUCTS /api/products

PRODUCTS /api/products

GET — Ambil Data

Semua produk

```
curl http://localhost:3000/api/products
```

Satu produk by ID

```
curl http://localhost:3000/api/products/1
```

Filter by kategori

```
curl http://localhost:3000/api/products?kategori=buah
```

Search by nama

```
curl http://localhost:3000/api/products?search=apel
```

Filter harga min-max

```
curl "http://localhost:3000/api/products?minHarga=5000&maxHarga=15000"
```

Sort by harga

```
curl http://localhost:3000/api/products?sort=harga
```

Sort by nama

```
curl http://localhost:3000/api/products?sort=nama
```

Kombinasi filter + search + sort

```
curl "http://localhost:3000/api/products?kategori=buah&sort=harga"
```

PRODUCTS /api/products

POST — Tambah Produk Baru

```
curl -X POST http://localhost:3000/api/products \
```

```
-H "Content-Type: application/json" \
```

```
-d '{
```

```
  "nama": "Mangga Harum Manis",
```

```
  "kategori": "buah",
```

```
  "harga": 12000,
```

```
  "stok": 75
```

```
}'
```

PUT — Update Penuh (semua field wajib dikirim)

```
curl -X PUT http://localhost:3000/api/products/1 \
```

```
-H "Content-Type: application/json" \
```

```
-d '{
```

```
  "nama": "Apel Fuji Premium",
```

```
  "kategori": "buah",
```

```
  "harga": 20000,
```

```
  "stok": 80
```

```
}'
```

PATCH — Update Sebagian (hanya field yang mau diubah)

Update harga saja

```
curl -X PATCH http://localhost:3000/api/products/1 \
```

```
-H "Content-Type: application/json" \
```

```
-d '{"harga": 20000}'
```

Update stok saja

PRODUCTS /api/products

Update stok saja

```
curl -X PATCH http://localhost:3000/api/products/1 \
  -H "Content-Type: application/json" \
  -d '{"stok": 50}'
```

Update nama dan harga

```
curl -X PATCH http://localhost:3000/api/products/1 \
  -H "Content-Type: application/json" \
  -d '{"nama": "Apel Premium", "harga": 18000}'
```

DELETE - Hapus Produk

```
curl -X DELETE http://localhost:3000/api/products/1
```

USERS /api/users

GET - Ambil Data

Semua user

```
curl http://localhost:3000/api/users
```

Satu user by ID

```
curl http://localhost:3000/api/users/1
```

USERS /api/users

POST - Tambah User Baru

```
curl -X POST http://localhost:3000/api/users \
  -H "Content-Type: application/json" \
  -d '{
    "nama": "Dina",
    "email": "dina@tazkia.ac.id",
    "role": "user"
  }'
```

PUT - Update User

```
curl -X PUT http://localhost:3000/api/users/2 \
  -H "Content-Type: application/json" \
  -d '{
    "nama": "Budi Santoso",
    "email": "budi@tazkia.ac.id",
    "role": "admin"
  }'
```

DELETE - Hapus User

```
curl -X DELETE http://localhost:3000/api/users/2
```